SENTIMENT ANALYSIS ON MOVIE REVIEWS USING MACHINE LEARNING

Machine Learning & Artificial Intelligence
Internship Project Report

Prepared By	Akhil Marada
Internship Organization	Cloudcredits Technologies
Project	Sentiment Analysis On Movie Reviews

Executive Summary

This project focuses on building a sentiment analysis system capable of classifying IMDb movie reviews as positive or negative. The work included dataset validation, text preprocessing, exploratory analysis, TF-IDF feature extraction, model comparison, evaluation, and deployment using Streamlit. Two machine learning models, Multinomial Naive Bayes and Logistic Regression, were evaluated. Logistic Regression achieved the best performance with 90.65% test accuracy and a positive-class F1-score of 90.82%, and was selected as the final deployed model.

The workflow covered dataset inspection, exploratory data analysis (EDA), duplicate removal, text preprocessing, TF-IDF feature extraction, stratified data splitting, model comparison, evaluation, model saving, and application development. The original dataset contained 50,000 reviews with an equal number of positive and negative labels. After removing 418 duplicate records and cleaning the text, 49,582 records remained.

1. Introduction

Sentiment analysis is a natural language processing (NLP) task that identifies the opinion or polarity expressed in text. It is commonly used to understand customer feedback, product reviews, social-media discussions, and other forms of user-generated content.

Movie reviews contain linguistic signals such as evaluative words, negation, and short phrases that communicate a reviewer's opinion. This project uses supervised machine learning to learn patterns from labelled IMDb reviews and classify unseen reviews into positive or negative sentiment.

2. Problem Definition

The objective is to predict whether a movie review expresses positive or negative sentiment based on its text.

Attribute	Description
Problem Type	Binary Text Classification
Input	Movie review text
Target Variable	sentiment
Target Classes	positive / negative
Output	Predicted sentiment and model confidence

Project Objectives

- Inspect and validate the movie-review dataset.
- Explore sentiment distribution, review length, text quality, and vocabulary.
- Clean HTML artifacts and normalize text while preserving negation.
- Transform review text into numerical TF-IDF features.
- Compare Multinomial Naive Bayes and Logistic Regression.
- Evaluate the selected model and integrate it into a Streamlit application.

3. Dataset Description and Preparation

The project uses the IMDb movie-review dataset stored as IMDB Dataset.csv. The file contains two columns: review and sentiment. Source: https://zenodo.org/records/7928582?

Dataset Attribute	Value
Original Records	50,000
Columns	2

Positive Reviews	25,000
Negative Reviews	25,000
Missing Values	0
Duplicate Records	418
Final Cleaned Records	49,582

Data Quality Observations

- HTML tags were found in 29,202 reviews (58.4% of the original dataset).
- Original review length averaged approximately 1,309 characters; the median was 970.
- The longest original review contained 13,704 characters.
- Duplicate records were removed before model training.

After duplicate removal and preprocessing, the final dataset contained 49,582 records with no missing values, duplicate records, empty cleaned reviews, or remaining HTML tags. The final class counts were 24,884 positive and 24,698 negative reviews.

Final Class	Records	Share
Positive	24,884	50.19%
Negative	24,698	49.81%
Total	49,582	100.00%

4. Exploratory Data Analysis

EDA was performed to understand the label distribution, review length, text characteristics, and vocabulary patterns before model development.

4.1 Sentiment Distribution

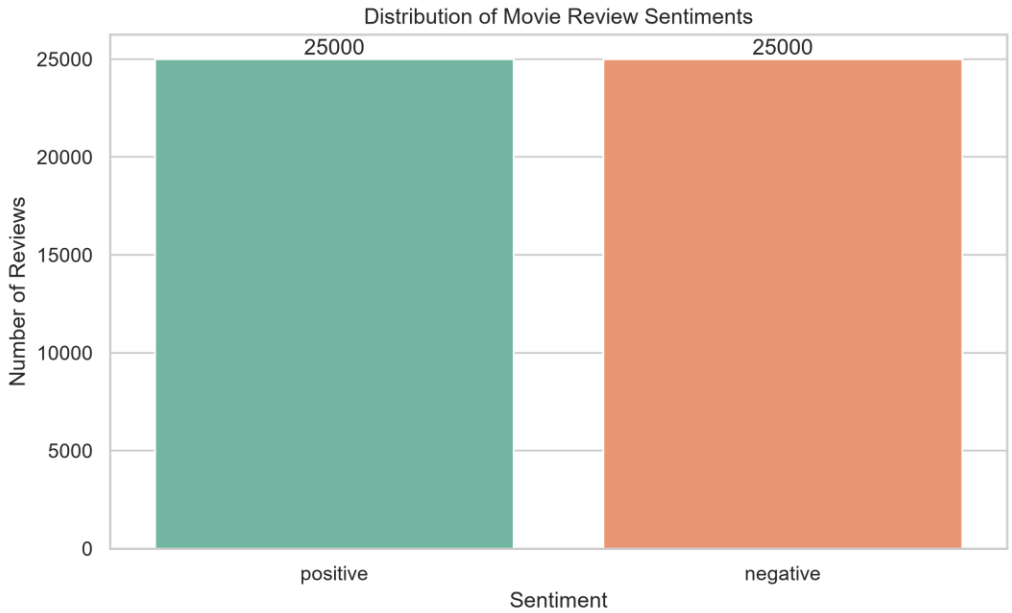


Figure 1: Sentiment Distribution in the IMDb Review Dataset

Observation: The original dataset is balanced, containing 25,000 positive and 25,000 negative reviews. This provides a balanced starting point for binary classification.

4.2 Review Length Distribution

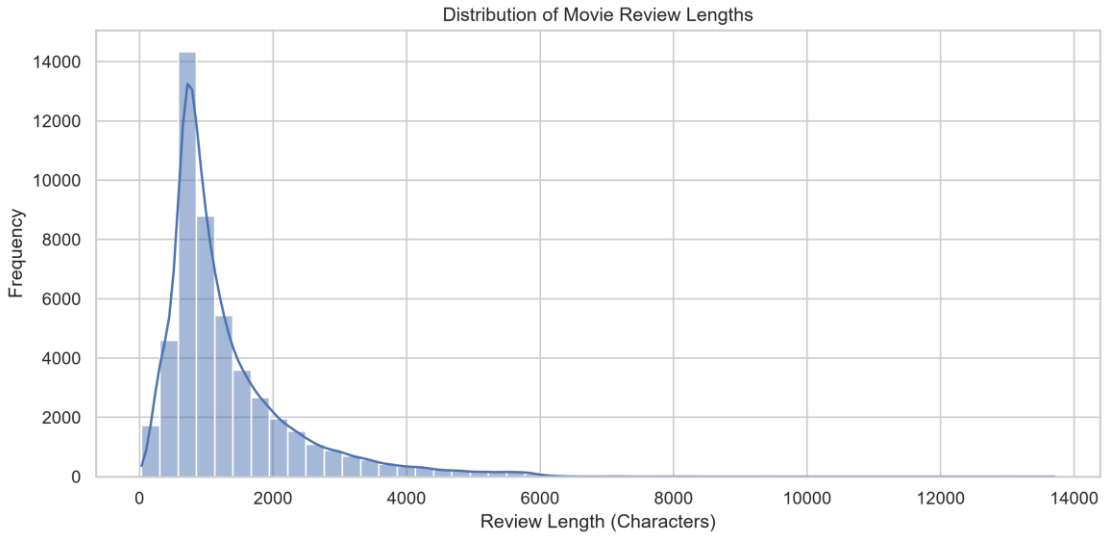


Figure 2: Distribution of Movie Review Lengths

Observation: Review lengths vary considerably. The distribution is right-skewed, with many shorter reviews and a smaller number of very long reviews.

4.3 Review Length by Sentiment

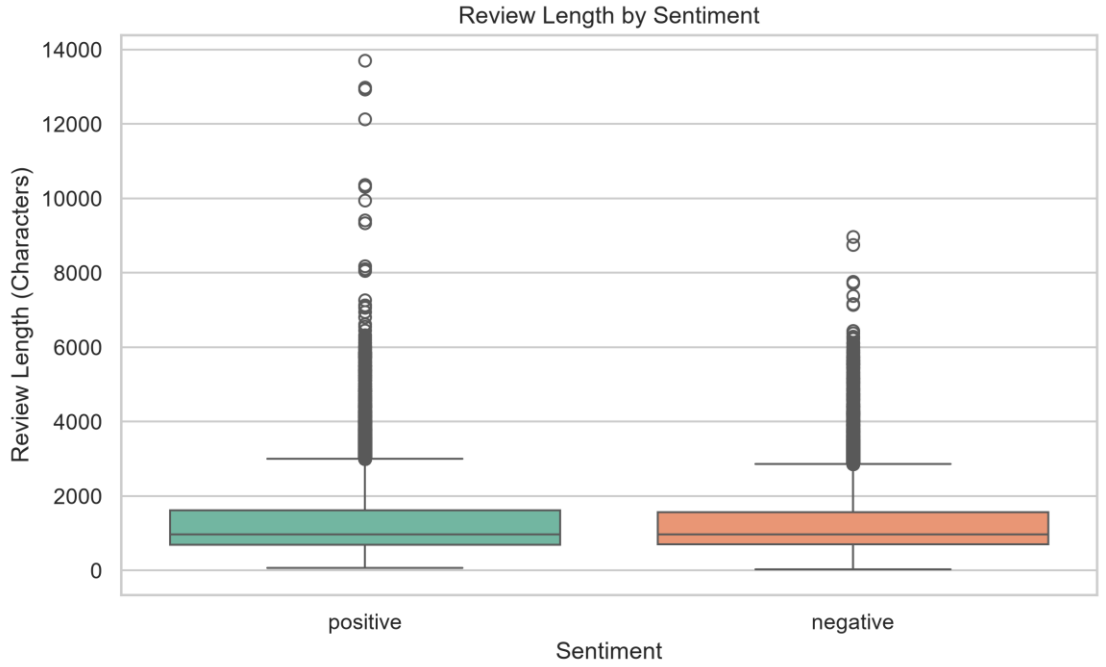


Figure 3: Review Length Distribution by Sentiment

Observation: Average review length was approximately 1,294 characters for negative reviews and 1,325 for positive reviews. Median lengths were approximately 973 and 968, respectively. The similar distributions suggest that review length alone is unlikely to be a strong sentiment predictor.

4.4 Frequent Words

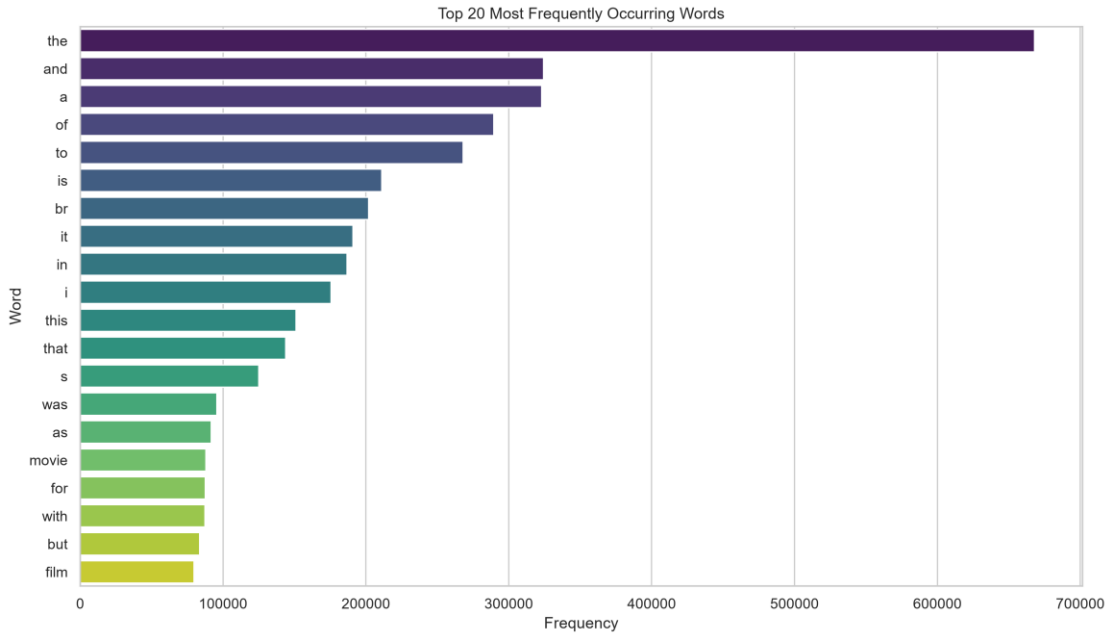


Figure 4: Twenty Most Frequent Tokens in the Review Text

Observation: Common English function words appear frequently. The token 'br' is associated with HTML line-break markup in the raw text, illustrating why HTML cleanup is important before feature extraction.

4.5 Positive and Negative Vocabulary Comparison

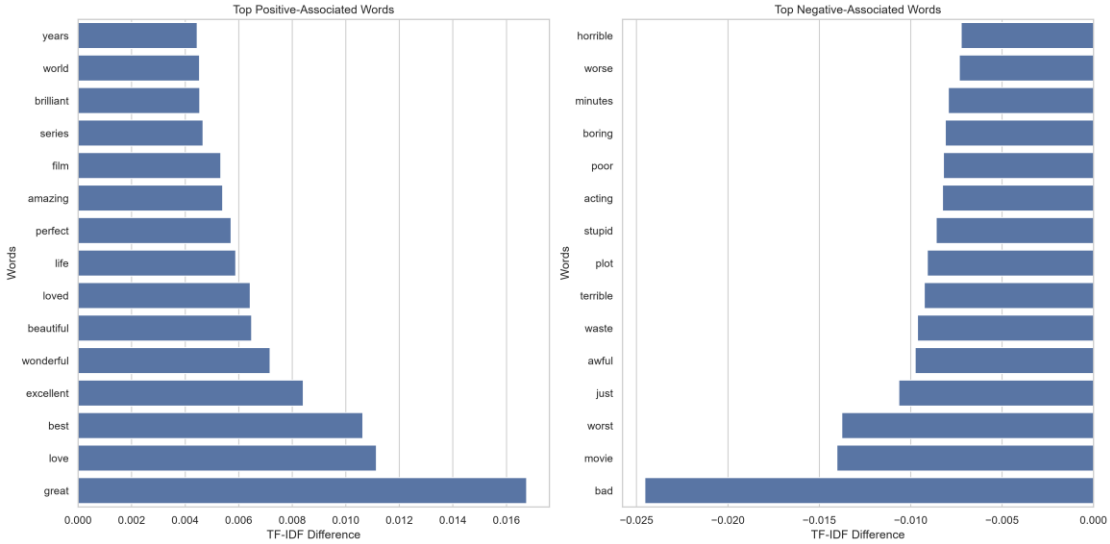


Figure 5: TF-IDF Vocabulary Comparison by Sentiment

Observation: Terms such as 'great', 'excellent', and 'wonderful' were associated with positive reviews in the exploratory sample, while terms such as 'awful', 'waste', 'terrible', and 'boring' were associated with negative reviews. These are exploratory associations and do not imply that any single term determines sentiment.

5. Text Preprocessing

A reusable clean_text() function was implemented in src/preprocess.py and applied to the review column. The preprocessing steps included:

- Handling non-string and empty input safely.
- Decoding HTML entities and removing HTML tags.
- Converting text to lowercase and normalizing apostrophes.
- Expanding selected contractions, such as “can't” to “can not”.
- Removing punctuation and special characters while retaining letters and numbers.
- Collapsing repeated whitespace.

Negation was deliberately preserved. For example, 'not good' must remain distinguishable from 'good'; removing the word 'not' could reverse the meaning. Hyphenated expressions are separated into tokens. The preprocessing function was validated with automated tests, including checks for HTML removal, contractions, negation, punctuation, whitespace, and non-string input.

6. Data Splitting and TF-IDF Feature Extraction

The cleaned dataset was split into training and testing partitions using an 80:20 stratified split with random_state=42.

Partition	Records	Positive	Negative
Training	39,665	19,907	19,758
Testing	9,917	4,977	4,940
Total	49,582	24,884	24,698

Text was converted into numerical features using scikit-learn's TfidfVectorizer. The configured parameters were max_features=50000, ngram_range=(1, 2), min_df=2, max_df=0.95, and sublinear_tf=True. Unigrams represent individual tokens, while bigrams represent two-token sequences and can capture short phrases such as 'not good'.

The vectorizer was included inside a scikit-learn Pipeline with each classifier. It was therefore fitted on training data within cross-validation and model fitting, rather than learning vocabulary from the complete dataset before the split.

7. Model Development

Two supervised machine-learning algorithms were selected for comparison:

7.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier commonly used for text classification. It provides a computationally efficient baseline for sparse text features.

7.2 Logistic Regression

Logistic Regression is a linear classification algorithm that estimates class probabilities from input features. It is a strong baseline for high-dimensional sparse TF-IDF representations. The model was configured with max_iter=1000 and random_state=42.

Both models used the same TF-IDF configuration to make the comparison consistent.

8. Model Evaluation

Five-fold StratifiedKFold cross-validation was conducted on the training partition with shuffle=True and random_state=42. Approximate cross-validation accuracy and macro F1 were 88.29% for Multinomial Naive Bayes and 90.58% for Logistic Regression.

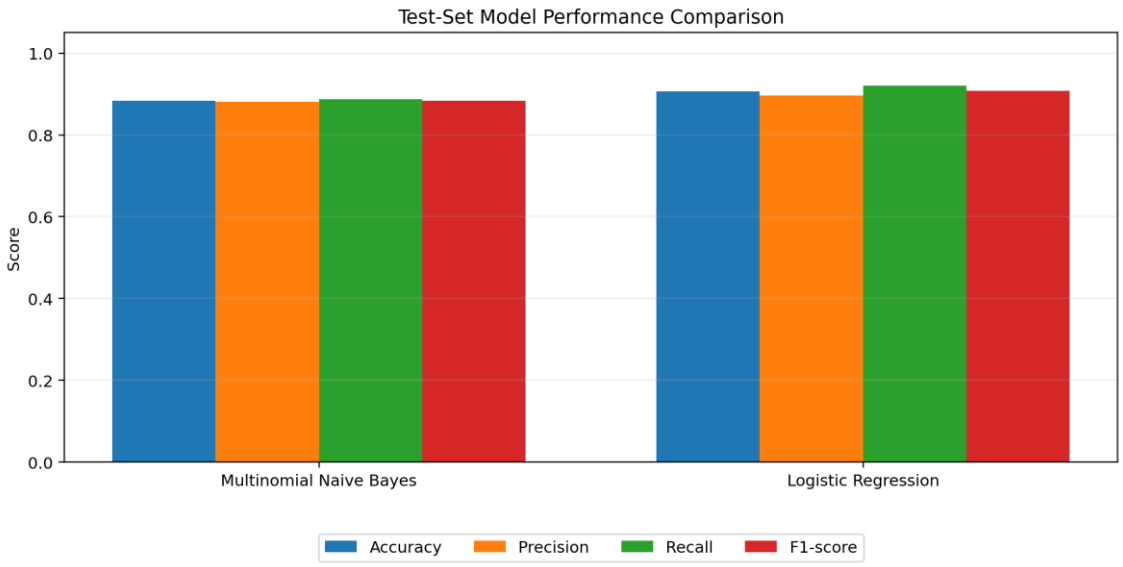


Figure 6: Test-Set Model Performance Comparison

Model	Accuracy	Precision*	Recall*	F1-score*
Multinomial Naive Bayes	88.32%	88.09%	88.73%	88.41%
Logistic Regression	90.65%	89.58%	92.08%	90.82%

* Precision, recall, and F1-score in this table are positive-class metrics. Accuracy is overall accuracy.

Observation: Logistic Regression outperformed Multinomial Naive Bayes on the reported test partition across the listed metrics. It achieved 90.65% accuracy and a 90.82% positive-class F1-score.

9. Confusion Matrix Analysis

9.1 Logistic Regression

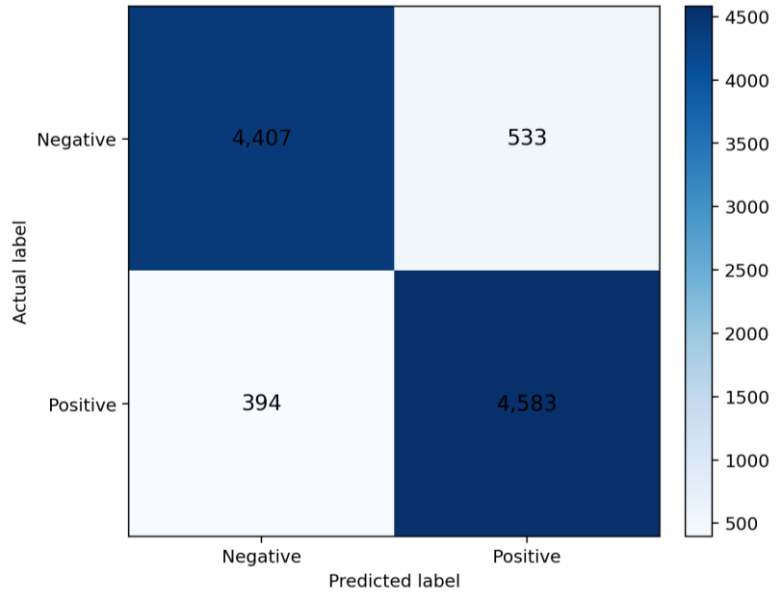


Figure 7: Logistic Regression Confusion Matrix

The matrix uses actual labels as rows [negative, positive] and predicted labels as columns [negative, positive]. Logistic Regression correctly classified 4,407 negative reviews and 4,583 positive reviews. It misclassified 533 negative reviews as positive and 394 positive reviews as negative.

9.2 Multinomial Naive Bayes

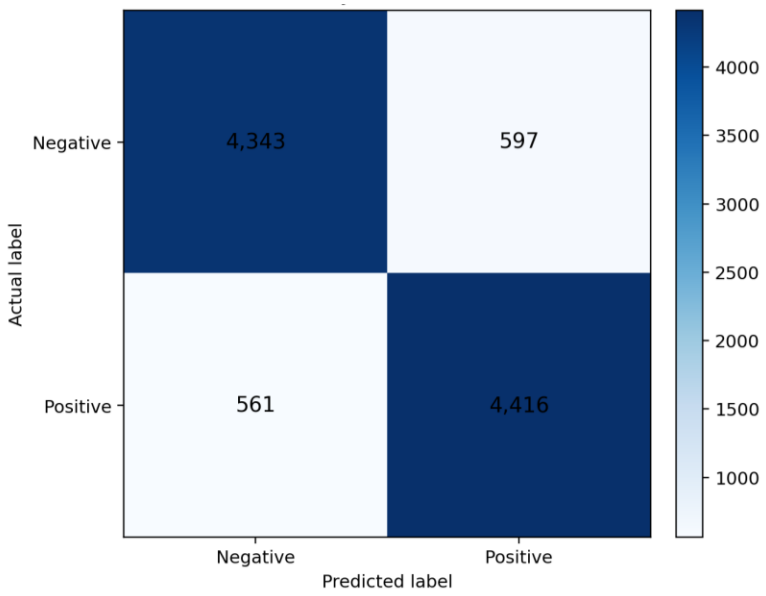


Figure 8: Multinomial Naive Bayes Confusion Matrix

Multinomial Naive Bayes correctly classified 4,343 negative reviews and 4,416 positive reviews. It misclassified 597 negative reviews as positive and 561 positive reviews as negative. Logistic Regression produced fewer errors in both directions on this test partition.

10. Final Model Selection

Logistic Regression was selected as the final model because it achieved stronger cross-validation results and stronger reported test-set metrics than Multinomial Naive Bayes.

Final Model Metric	Result
Selected Algorithm	Logistic Regression
Test Accuracy	90.65%
Positive-Class Precision	89.58%
Positive-Class Recall	92.08%
Positive-Class F1-score	90.82%
TF-IDF Vocabulary Size	50,000

The fitted Logistic Regression pipeline, including the TF-IDF vectorizer, was saved as models/final_sentiment_model.pkl. Saving the complete pipeline ensures that incoming reviews use the same fitted text representation during inference.

11. Streamlit Application Deployment

The selected model was integrated into a Streamlit application (app.py) to provide an interactive sentiment-analysis interface. The application has three main tabs.

11.1 Single Review Analysis

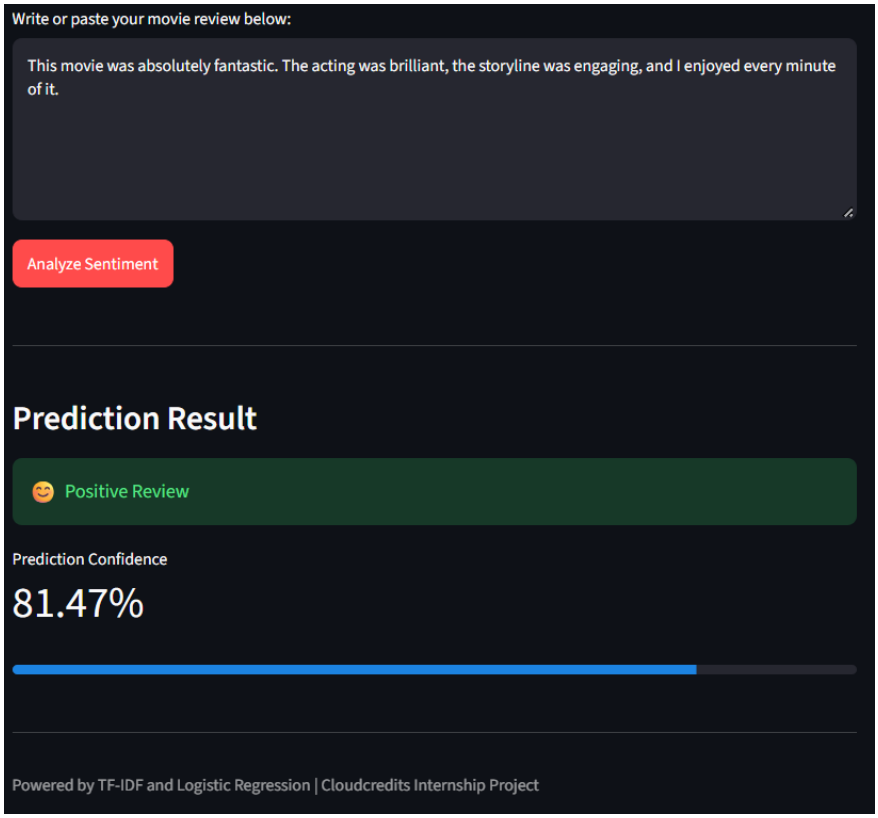


Figure 9: Single Review Sentiment Analysis Interface

Users can enter or select a sample movie review. The application applies the shared clean_text() preprocessing function, predicts the sentiment, and displays the model's confidence.

11.2 Batch Review Analysis

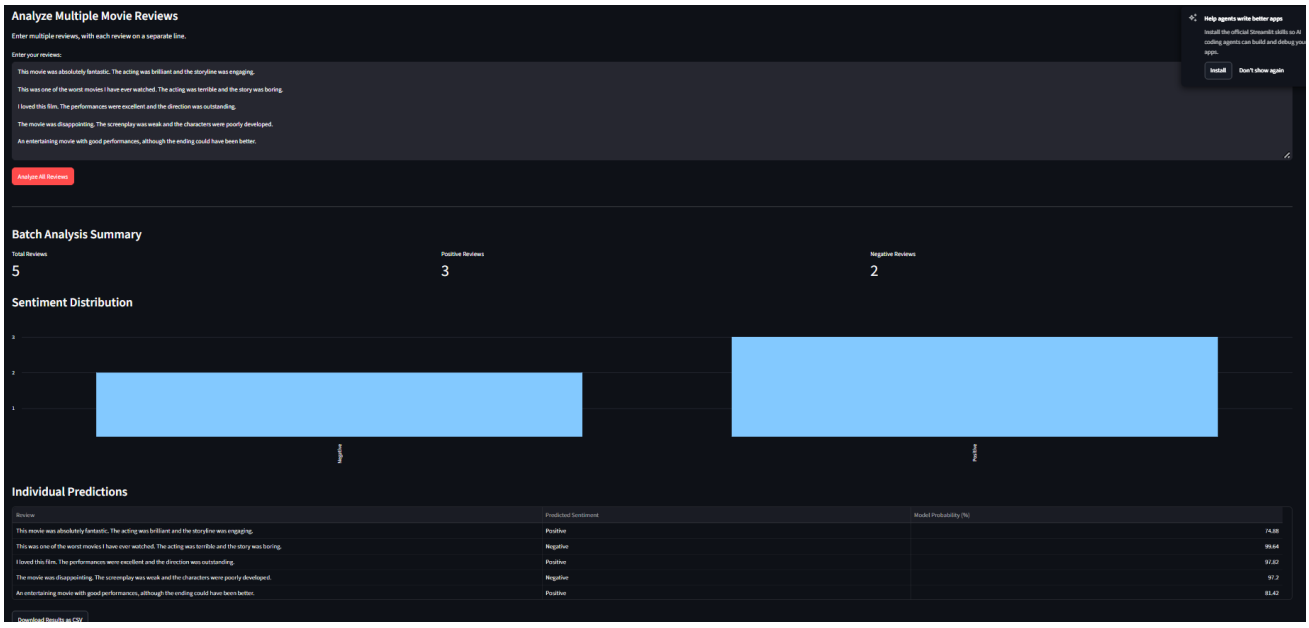


Figure 10: Batch Review Analysis Interface

Users can submit multiple reviews, one per line. The application predicts each review, summarizes positive and negative counts, presents a chart and results table, and supports downloading the predictions as a CSV file.

11.3 About Model

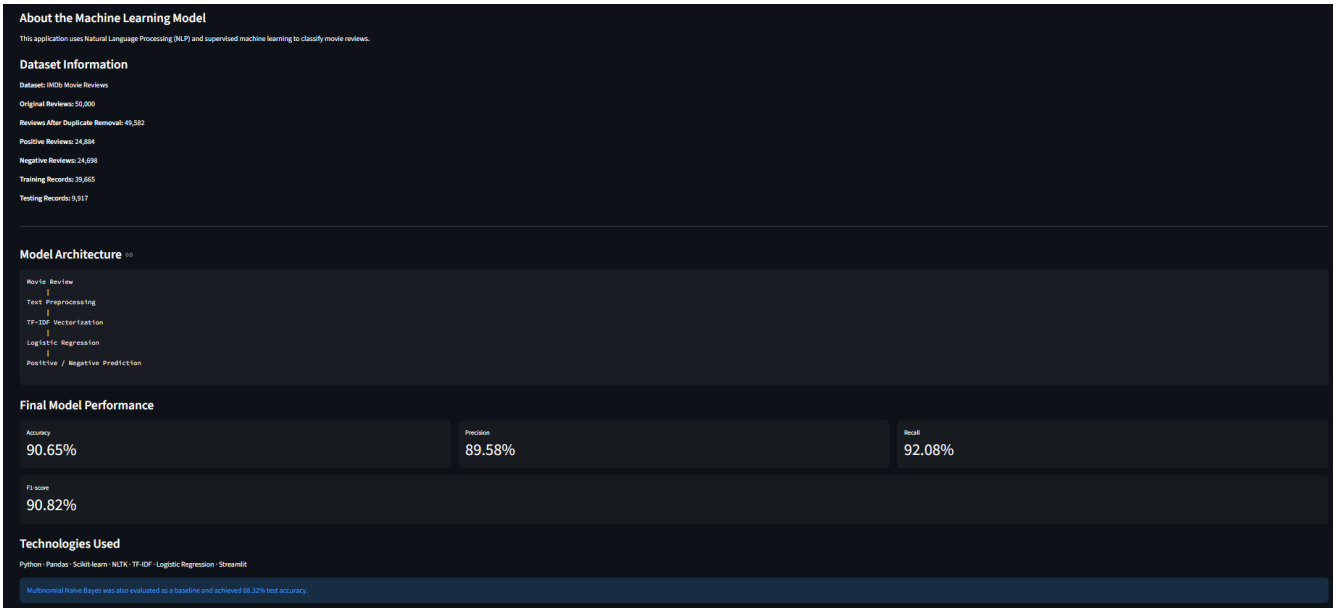


Figure 11: Model Information Interface

The About Model tab summarizes the dataset, model architecture, evaluation metrics, and technologies used. The application also applies input safeguards, including a maximum review length and a maximum batch size.

The same clean_text() function is used during training-data preparation and application inference, helping maintain consistency between model training and real-time predictions.

12. Project Structure

Path	Purpose
data/raw/IMDB Dataset.csv	Original IMDb review dataset
data/processed/cleaned_reviews.csv	Deduplicated and cleaned dataset
data/processed/train_reviews.csv	Training partition
data/processed/test_reviews.csv	Testing partition
notebooks/01_Exploratory_Data_Analysis.ipynb	EDA notebook
src/preprocess.py	Reusable text-cleaning function
src/process_dataset.py	Dataset cleaning workflow
src/split_dataset.py	Stratified train/test split
src/compare_models.py	Cross-validation model comparison
src/train_model.py	Model training
src/evaluate_models.py	Model evaluation
src/save_final_model.py	Final model persistence
models/final_sentiment_model.pkl	Saved Logistic Regression pipeline
app.py	Streamlit application
images/	EDA charts and application screenshots
requirements.txt	Python dependencies

13. Challenges Faced

- Large number of HTML tags present in raw reviews required additional cleaning.
- Selection of preprocessing rules needed careful handling because removing negation words such as "not" could alter sentiment meaning.
- Review lengths varied significantly, requiring TF-IDF configuration that could handle both short and very long reviews.
- Comparing models using consistent preprocessing pipelines was important to avoid unfair evaluation.

14. Limitations and Future Scope

Limitations

- The task is limited to binary positive/negative sentiment classification.
- The model may struggle with sarcasm, subtle or mixed opinions, spelling errors, and context-dependent meaning.
- The dataset contains English-language IMDb reviews and may not generalize to other domains or languages.
- The test-set results were inspected while comparing models and selecting the final model. Therefore, these metrics are not a completely untouched independent estimate after model selection.

Future Scope

- Use training-only cross-validation or a validation set for model selection, then evaluate once on a fresh untouched test set.
- Perform broader hyperparameter tuning and error analysis of misclassified reviews.
- Compare additional algorithms such as Linear Support Vector Machines.
- Evaluate probability calibration and test generalization on external review datasets.
- Explore transformer-based models as a separate extension.

15. Conclusion

This project implemented an end-to-end traditional NLP and machine-learning workflow for movie-review sentiment classification. The work included dataset quality checks, exploratory analysis, text preprocessing, TF-IDF feature extraction, cross-validated model comparison, test evaluation, model persistence, and an interactive Streamlit application.

Among the evaluated algorithms, Logistic Regression achieved the strongest reported performance, with 90.65% accuracy and 90.82% positive-class F1-score. The application supports both single-review and batch sentiment analysis. The project also demonstrates the importance of preserving negation, fitting text transformations only on training data, applying consistent preprocessing during inference, and documenting evaluation limitations.